Software Requirements Specification (SRS)

ORBIT.CyberGraph-Node

Team Number: 45

Documented by: Anushka Sinha

Client: Satya Vithala – NJSecure

Team Members

Anushka Sinha

Ayush Agarwal

Masthan Sharif Shaik

Naveen Kumar Medam

Yashav Bhatnagar

February 19, 2026

Brief Problem Statement

Small and medium banking environments rely heavily on SIEM platforms such as Wazuh but lack sufficient contextual intelligence to prioritize threats based on data sensitivity and business impact. As a result, alerts are often treated uniformly, leading to alert fatigue and delayed responses. ORBIT.CyberGraph-Node addresses this problem by enriching Wazuh with privacy-aware context, local knowledge graph reasoning, and human-validated defensive actions sourced from ORBIT Core.

System Requirements

- Ubuntu-based Linux operating system
- Docker and Docker Compose
- Wazuh SIEM (Manager, API, and Active Response)
- Microsoft Presidio Analyzer
- Neo4j Graph Database
- Neo4j Bloom
- ORBIT Core (external system)
- REST-based APIs

The system operates locally and does not transmit raw telemetry, logs, or sensitive customer data externally.

Users Profile

- **SOC Analyst:** These are highly technical security professionals responsible for reviewing alerts and approving mitigation actions. They have advanced familiarity with SIEM platforms like Wazuh and are proficient in incident response workflows.
- **System Administrator:** Responsible for overseeing system operations, including the maintenance of the Ubuntu-based OS and Docker environment. They possess expert-level knowledge of Linux systems, container orchestration, and network configuration.
- **ORBIT Core:** While an external system rather than a human, it acts as a primary actor that identifies threats and generates Action Cards. It interacts with the Node via REST-based APIs in a fully automated, machine-to-machine mode..

Feature Requirements (Use Cases)

No.	Use Case Name	Description	Release
1	Local Knowledge Graph Creation	Builds and maintains a local knowledge graph representing assets, software, and vulnerabilities within the monitored environment. This graph provides contextual awareness required for risk analysis and decision-making.	R1
2	Privacy Scan & Crown Jewel ID	Scans local data stores to identify sensitive information using privacy detection techniques. Assets containing high-value or regulated data are classified as crown jewels for prioritization.	R1
3	Subscription Optimization	Generates optimized subscription requests that determine what Action Cards the Node should receive from the ORBIT Core. This reduces unnecessary communication while preserving relevance and privacy.	R1
4	Inject Action Card into Wazuh	Receives Action Cards from the ORBIT Core and translates them into native Wazuh alerts. This enables analysts to review recommended actions within existing security workflows.	R1
5	HITL Execution	Ensures all mitigation actions are reviewed and explicitly approved by a human analyst. Approved actions are executed using Wazuh Active Response mechanisms.	R1
6	Pending Action Assignment	Supports deferred mitigation workflows by assigning unresolved actions to analysts. Pending actions are tracked until completion or resolution.	R1

7	Differential Updates	Communicates incremental changes in the local environment to the ORBIT Core. Only deltas are transmitted to reduce data volume and protect sensitive information.	R2
8	Completion Feedback	Reports the outcome of executed actions back to the ORBIT Core. Feedback enables learning, metrics collection, and continuous improvement of recommendations.	R2

Use Case Diagram

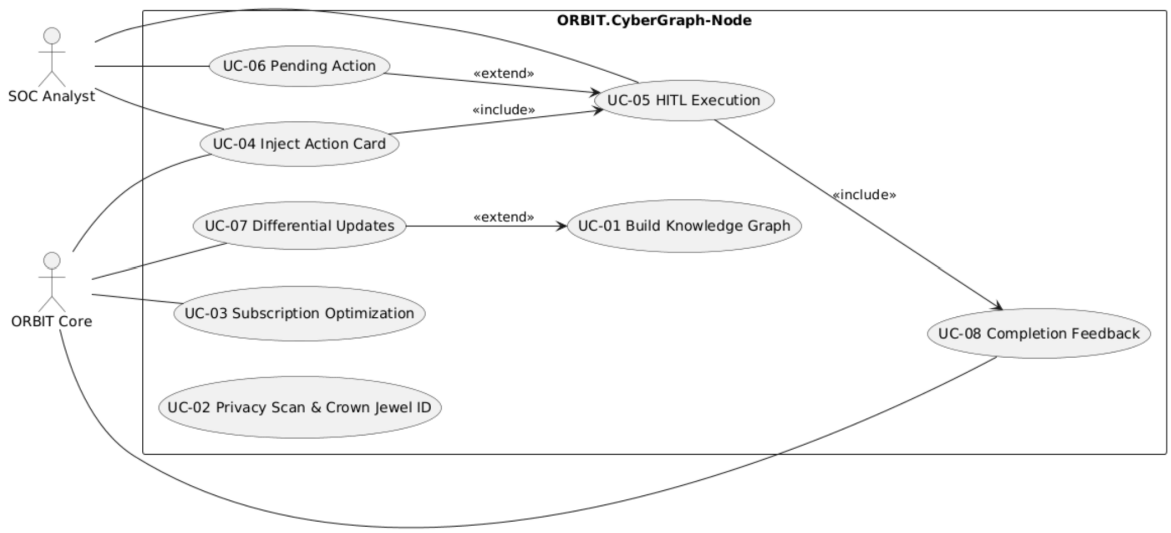

Figure 1: Use Case Diagram for ORBIT.CyberGraph-Node

Use Case Descriptions

Use Case Number	UC-01
Use Case Name	Local Knowledge Graph Creation

Overview	This use case describes how the ORBIT Node constructs and maintains a local knowledge graph of assets, software, and vulnerabilities. The graph provides contextual awareness required for risk reasoning and downstream decision-making. Updates occur periodically or when environmental changes are detected.
Actors	Node Agent, Wazuh
Pre-condition	Wazuh agents are active and reporting telemetry.
Flow	**Main Flow:** 1. Retrieve asset and vulnerability data from Wazuh. 2. Normalize and map data to ORBIT schema. 3. Update Neo4j graph. **Alternate Flow:** • Wazuh unavailable → retry later.
Post-condition	Knowledge graph reflects current environment.

Use Case Number	UC-02
Use Case Name	Privacy Scan & Crown Jewel Classification
Overview	This use case identifies sensitive data stored within local assets using privacy scanning techniques. Based on detected data types and distribution, assets are classified by sensitivity and high-value targets are marked as crown jewels. This process runs automatically without human intervention.
Actors	Node Agent, Presidio
Pre-condition	Scan targets are accessible.

Flow	**Main Flow:** 1. Select files or database exports. 2. Analyze text with Presidio. 3. Assign sensitivity scores. **Alternate Flow:** • If a scan target is inaccessible, the asset is skipped and logged for later analysis.
Post-condition	Assets classified by sensitivity.

Use Case Number	UC-03
Use Case Name	Subscription Optimization
Overview	This use case generates optimized subscription requests that determine what types of Action Cards the ORBIT Core should send to the Node. Subscriptions are derived from local context to minimize unnecessary communication. This ensures relevance while preserving privacy.
Actors	Node Agent, ORBIT Core
Pre-condition	Knowledge graph exists.
Flow	**Main Flow:** 1. Analyze local context. 2. Generate subscription summary. 3. Send subscription to Core. **Alternate Flow:** • If the ORBIT Core is unreachable, the subscription update is queued locally and retried later without impacting local Node operations.
Post-condition	Core subscription updated.

Use Case Number	UC-04
Use Case Name	Inject Action Card into Wazuh
Overview	This use case handles the reception and presentation of Action Cards generated by the ORBIT Core. The Node translates these recommendations into native Wazuh alerts so that analysts can review them within familiar workflows. No actions are executed automatically.
Actors	Node Agent, ORBIT Core, Wazuh
Pre-condition	Action Card received from Core.
Flow	**Main Flow:** 1. Validate Action Card. 2. Translate to Wazuh format. 3. Inject alert. **Alternate Flow:** • If the Action Card fails validation, it is discarded and an error is logged.
Post-condition	Alert visible to analyst.

Use Case Number	UC-05
Use Case Name	Human-in-the-Loop Execution
Overview	This use case ensures that all defensive actions remain human-approved. The SOC analyst reviews contextual information and explicitly authorizes execution using Wazuh Active Response mechanisms. This maintains human-in-the-loop control over mitigations.
Actors	SOC Analyst, Wazuh
Pre-condition	ORBIT alert visible in Wazuh.

Flow	**Main Flow:** 1. Analyst reviews alert. 2. Analyst approves execution. 3. Action executed via Active Response. **Alternate Flow:** • If the analyst rejects the action, execution is aborted and the decision is recorded.
Post-condition	Action executed and logged.

Use Case Number	UC-06
Use Case Name	Pending Action Assignment
Overview	This use case supports deferred or multi-step mitigation workflows. When an action cannot be completed immediately, it is assigned as a pending task to an analyst. Progress and resolution status are tracked over time.
Actors	SOC Analyst
Pre-condition	Action pending resolution.
Flow	**Main Flow:** 1. Create pending action. 2. Assign to analyst. 3. Track status.
Post-condition	Action assigned and tracked.

Use Case Number	UC-07
Use Case Name	Differential Updates

Overview	This use case communicates incremental environmental changes from the Node to the ORBIT Core. Only deltas are transmitted to reduce data volume and preserve privacy. Updates are triggered by changes in assets, vulnerabilities, or sensitivity.
Actors	Node Agent, ORBIT Core
Pre-condition	Local context changed.
Flow	**Main Flow:** 1. Detect changes. 2. Package deltas. 3. Send update to Core. **Alternate Flow:** • If the ORBIT Core is temporarily unavailable, differential updates are cached locally and synchronized when connectivity is restored.
Post-condition	Core context synchronized.

Use Case Number	UC-08
Use Case Name	Completion Feedback
Overview	This use case reports the outcome of executed actions back to the ORBIT Core. Feedback includes execution status and relevant metadata but excludes sensitive raw data. This enables learning and metrics at the Core level.
Actors	Node Agent, ORBIT Core
Pre-condition	Action executed.

Flow	**Main Flow:** 1. Capture execution result. 2. Update knowledge graph. 3. Notify Core. **Alternate Flow:** • If feedback delivery to the ORBIT Core fails, execution results are retained locally and transmitted when connectivity is restored.
Post-condition	Core receives execution feedback.